Confidential mode: what a Routstr node can and cannot see

The short version. In confidential mode, your routstrd talks to the AI provider (for example Venice) through the Routstr node. The node only relays encrypted traffic and adds its own API key. It cannot read your prompts or the answers, and it cannot change them. Billing uses the provider's own usage report. You need no new apps, accounts or special hardware (no enclaves): it is a software upgrade of routstrd and the node, switched on per request. Since protocol v3 the costly setup is paid once per connection to the provider, not once per request.

What you trust, and for what

Party	You trust it for	You no longer trust it for
The provider (e.g. Venice)	Running the model you asked for and handling your data per its policy; it sees your prompt, as always.	
The Routstr node	Business honesty: taking your payment, relaying traffic, refunding unused money, not cutting connections short. Reputation covers this, as it does today.	Reading your prompts and answers, changing them, swapping the model, billing more than the provider's usage report for a completed request.
Your routstrd	Everything: it holds the keys, on your machine.	

The assumption behind this design: **nodes resell reputable providers, and you should not have to trust the node much.** Cryptography replaces most of that trust; node reputation covers the rest.

The three common objections

1. "Can the node change my tool calls, system prompt or messages?" No. Your request body (messages, system prompt, tools, tool results) is encrypted with keys only your routstrd has; if the node changed one byte, the provider would reject it. Likewise your routstrd checks every piece of the answer, tool calls included, so a modified answer is detected and discarded. The node writes only at the start of a connection, before the keys switch to ones only your routstrd holds: the opening with its API key, which a zero-knowledge proof shows matches a published template. It could still write other or extra records there unseen (another account, a header asking the provider to retain data). None of them carries or changes your content, but they could change how the provider handles it where its API allows that.

2. "Can the node change the model, say to a cheaper one, or bill me for a dearer one?" No, with one exception for failed requests (under Money). The model name and token limits sit at the end of your encrypted request, where they override anything earlier. The node holds back your request until your routstrd has proven that its public parts (the request line and the ending that names the agreed model and limits) are what was agreed; until then the provider has an incomplete request and does nothing. Your routstrd also checks the provider's certificate and that its hostname is on your trusted list. The bill is the provider's own usage report, which your routstrd proves to the node, at the rates the node signed beforehand; routstrd rejects a signed receipt that does not match.

3. "Can the Routstr node read my prompts and responses?" No. It only handles ciphertext; the keys never leave your machine. It still sees which provider and model you use, when, how long your request is, which of your requests share a connection, and the size of each streamed chunk, which can leak some of an answer (research has partially reconstructed responses from it), far less than without this mode, when it sees everything. On a provider error it sees only the status and headers.

What it costs, and the limits

- **Speed and data:** opening a connection runs a two-party computation with the node: routstrd downloads about 16 MB and uploads about 1.1 MB (v2: 22 MB and 1.2 MB, for every request). Where the node offers HTTP/2, routstrd keeps the connection and sends your next requests on it: each moves about 0.5-1 MB down and under 0.1 MB up, and the first token came in 0.5-1.3 s in our tests (2 s on the request that opened the connection; one host, so add your network). Parallel requests open more connections. Where the node offers only HTTP/1.1, every request pays the opening; routstrd prepares connections in the background while you send requests.
- **Money:** from your usual balance with that node, at the rates it signed. Opening a connection holds a small deposit (by default at least 1 sat), returned with your first settled request and kept if you open a connection and never use it. Per request the node holds back the maximum (a full-context prompt, your token limit and a small margin) and charges the usage the provider reported (an image: the signed flat price). The offer may add a session fee, once per connection, and an abandon fee for a request dropped before it reached the provider. A provider refusal costs the session fee for a 429, 503 or 401-403, the abandon fee for any other 4xx. If the provider got your request but no usage report comes back (another 5xx, a cut stream), the node charges that request's whole reservation; you cannot prove who cut it.
- **Providers:** chat and images; tested with one provider, Venice (others need checking). If this mode is unavailable, a confidential request is refused, never sent openly. routstrd does not use a node that still runs the older v2, and says so.
- **Not covered:** the provider still sees your prompt, and can tell that requests on one connection come from one client (routstrd keeps separate connections per local API key). The node can refuse service or keep money after a failed request (reputation protects you).

The two-party computation and the HTTP/2 rules are new, have no published proof and need external review; a cheating node could learn at most 3 bits per connection about single-use values, if its semi-honest first step leaks no more.